DCN · STUDY GUIDE

The Application Layer

Chapter 2 — Computer Networking: A Top-Down Approach (8th Ed., Kurose & Ross)

Covers: Network application principles · HTTP & the Web · electronic mail (SMTP/IMAP) · DNS · P2P file distribution & BitTorrent · video streaming & CDNs · socket programming (UDP/TCP)

Format: Definitions, key rules, worked examples, watch-outs, and intuition boxes — followed by a consolidated quick-reference sheet

Reorganized for exam-ready review

Table of Contents

2.1 Principles of Network Applications

Definition **Network Application** — a set of programs running on different end systems that communicate with each other over the network. Application software runs only on **end systems** — not on network-core devices such as routers or link-layer switches.
Definition **Application Architecture** — designed by the application developer; dictates how the application is structured over the various end systems. Distinct from **network architecture** (the fixed, five-layer Internet architecture).
Definition **Client-Server Architecture** — an always-on host called the **server** services requests from many other hosts called **clients**. Clients do not directly communicate with each other. The server has a fixed, well-known IP address. Examples: Web, FTP, Telnet, e-mail.
Definition **Data Center** — a large collection of hosts used to create a powerful virtual server when a single server host cannot handle the load. Google has 19 data centers distributed around the world.
Definition **P2P Architecture** — minimal (or no) reliance on dedicated servers. The application exploits direct communication between pairs of peers (intermittently connected hosts controlled by users). Example: BitTorrent.
Definition **Self-scalability (P2P)** — each peer generates workload by requesting files but also adds service capacity by distributing files to other peers.
Definition **Process** — a program running within an end system. Processes on different hosts communicate by exchanging messages across the network. **Client process** initiates the communication at the start of a session; **server process** waits to be contacted to begin the session.
Definition **Socket** — the software interface between the application layer and the transport layer within a host. A process sends/receives messages through its socket. Also called the **Application Programming Interface (API)** between the application and the network.
Definition **IP Address** — a 32-bit quantity that uniquely identifies a host. **Port Number** — identifies which process (socket) in a destination host a packet should be delivered to. Web server: port 80. SMTP mail server: port 25.

Application Traffic Characteristics

- **Reliable data transfer** — guarantees all data sent is delivered correctly and completely
- **Loss-tolerant applications** — can tolerate some data loss without significant impairment (e.g., multimedia audio/video)
- **Bandwidth-sensitive applications** — have specific throughput requirements (e.g., Internet telephony at 32 kbps)
- **Elastic applications** — can use as much or as little throughput as available (e.g., e-mail, file transfer, Web transfers)

Definition **TLS (Transport Layer Security)** — an enhancement of TCP implemented in the application layer that provides encryption, data integrity, and end-point authentication. Not a third transport protocol — TLS code must be included on both client and server sides.
Definition **Application-Layer Protocol** — defines how an application's processes, running on different end systems, pass messages to each other. Defines: (1) types of messages exchanged, (2) syntax of message types, (3) semantics of fields, (4) rules for when and how messages are sent and responded to.

TCP vs. UDP Services

TCP SERVICES	UDP SERVICES
Connection-oriented — handshaking before app messages flow; full-duplex; must be torn down when finished	**Connectionless** — no handshaking before communication begins
Reliable data transfer — all data delivered without error, in proper order, no missing/duplicate bytes	**Unreliable data transfer** — no guarantee message arrives; may arrive out of order
Congestion-control mechanism — throttles sender when network is congested; fair-share bandwidth limiting	**No congestion control** — sender can pump data at any rate
Neither provides throughput guarantees or timing guarantees.	

APPLICATION	DATA LOSS	THROUGHPUT	TIME-SENSITIVE
File transfer/download	No loss	Elastic	No
E-mail	No loss	Elastic	No
Web documents	No loss	Elastic (few kbps)	No
Internet telephony / video conferencing	Loss-tolerant	Audio: few kbps–1 Mbps; Video: 10 kbps–5 Mbps	Yes: 100s of msec
Streaming stored audio/video	Loss-tolerant	Same as above	Yes: few seconds
Interactive games	Loss-tolerant	Few kbps–10 kbps	Yes: 100s of msec
Smartphone messaging	No loss	Elastic	Yes and no

APPLICATION	APP-LAYER PROTOCOL	TRANSPORT PROTOCOL
Electronic mail	SMTP [RFC 5321]	TCP
Remote terminal access	Telnet [RFC 854]	TCP
Web	HTTP 1.1 [RFC 7230]	TCP
File transfer	FTP [RFC 959]	TCP
Streaming multimedia	HTTP (e.g., YouTube), DASH	TCP
Internet telephony	SIP, RTP, or proprietary (Skype)	UDP or TCP

Intuition TCP is chosen for e-mail, Web, and file transfer because reliable delivery is essential. Internet telephony often prefers UDP to avoid TCP's congestion-control overhead — but because many firewalls block UDP, telephony apps often fall back to TCP.

Watch Out Neither TCP nor UDP provides throughput or timing guarantees. Time-sensitive applications must cope with this limitation through clever design, not transport-layer promises.

2.2 The Web and HTTP

The World Wide Web arrived in the early 1990s — the first Internet application to catch the general public's eye. It operates on demand: users receive what they want, when they want it.

Definition **Web Page (Document)** — consists of objects. An **object** is a file (HTML, JPEG, JavaScript, CSS, video clip) addressable by a single URL.
Definition **URL** — has two components: (1) the hostname of the server housing the object, and (2) the object's path name.
Example: `http://www.someSchool.edu/someDepartment/picture.gif` — hostname is `www.someSchool.edu`, path name is `/someDepartment/picture.gif`.
Definition **HTTP (HyperText Transfer Protocol)** — the Web's application-layer protocol [RFC 1945, 7230, 7540]. Implemented in client (browser) and server programs.
Definition **Stateless Protocol** — HTTP maintains no information about clients. If a client asks for the same object twice, the server resends it without any memory of the prior request.
Connection Types

- **Non-persistent connections** — each request/response pair sent over a separate TCP connection. Used in HTTP/1.0. Each connection transports exactly one request and one response.
- **Persistent connections** — all requests and corresponding responses sent over the same TCP connection. Default mode of HTTP/1.1. Supports **pipelining**.

Definition **RTT (Round-Trip Time)** — the time for a small packet to travel from client to server and back. Includes propagation, queuing, and processing delays.

```
Formula — Non-Persistent HTTP Response Time Total response time = 2 × RTT + transmission time of the file
(1 RTT for TCP three-way handshake, 1 RTT for the HTTP request/response)
```

Definition **Cookies** — technology with four components: (1) a cookie header line in the HTTP response, (2) a cookie header line in the HTTP request, (3) a cookie file on the user's end system managed by the browser, (4) a back-end database at the Web site [RFC 6265].
Worked Example — Cookies

1. Susan visits Amazon.com for the first time. Amazon creates a unique ID (e.g., 1678) and an entry in its back-end database.
2. Amazon's HTTP response includes: `Set-cookie: 1678`
3. Susan's browser appends a line to its cookie file: `amazon.com → 1678`
4. All subsequent HTTP requests to Amazon include: `Cookie: 1678`
5. Amazon can now track Susan's activity, maintain her shopping cart, and provide one-click shopping.

Definition **Web Cache (Proxy Server)** — a network entity that satisfies HTTP requests on behalf of an origin Web server. Has its own disk storage; acts as both server (to browsers) and client (to origin servers). Typically purchased/installed by an ISP.
Definition **Hit Rate** — the fraction of requests satisfied by a cache (typically 0.2 to 0.7 in practice).
Definition **Conditional GET** — an HTTP request that is (1) a GET method AND (2) includes an `If-Modified-Since:` header line. Allows a cache to verify its objects are up to date without re-fetching unchanged content.

```
Formulas Traffic intensity = (request rate × object size) / link bandwidth
Average delay (hit rate h) = h × (delay via cache) + (1−h) × (delay via origin server)
```

Worked Example — Web Caching Setup: institutional network on a 15 Mbps access link (100 Mbps LAN internally); average object size 1 Mbit; average request rate 15 req/sec; Internet delay (origin-side) 2 sec average.

- Traffic intensity (no cache) = (15 × 1) / 15 = **1.0** → delays become very large (minutes) — unacceptable
- **Solution 1 — upgrade link to 100 Mbps:** intensity drops to 0.15 → negligible access delay; total response time ≈ 2 sec (Internet delay). Cost: expensive link upgrade.
- **Solution 2 — install a Web cache (hit rate = 0.4):** 40% served by cache (~10 ms); 60% go to origin, access-link intensity = 0.6 (small delay, ~tens of ms). Average delay = 0.4(0.01) + 0.6(2.01) ≈ **1.2 seconds** — better than Solution 1!

Worked Example — Conditional GET **Step 1** — Cache requests object: `GET /fruit/kiwi.gif HTTP/1.1`
Step 2 — Server responds `200 OK` with `Last-Modified: Wed, 9 Sep 2015 09:23:24`. Cache stores object AND date.
Step 3 — A week later, another browser requests the same object; cache sends a conditional GET with `If-modified-since: Wed, 9 Sep 2015 09:23:24`.
Step 4 — If unmodified, server replies `304 Not Modified` (empty body). Cache serves its stored copy — no bandwidth wasted re-sending the object.

HTTP/2 and HTTP/3

Definition **HTTP/2** [RFC 7540, 2015] — goals: reduce perceived latency via request/response multiplexing over a single TCP connection, request prioritization, server push, and efficient header compression. Does **not** change HTTP methods, status codes, URLs, or header fields — only how data is formatted and transported.
Definition **HOL (Head of Line) Blocking** — in HTTP/1.1, a large object (e.g., video clip) at the head of the queue blocks small objects behind it from being sent. HTTP/1.1 browsers work around this by opening multiple parallel TCP connections (up to 6).
Definition **HTTP/2 Framing** — breaks each message into small **frames** and interleaves request and response messages on the same TCP connection. A header becomes one frame; a body becomes one or more frames. Frames are interleaved, binary-encoded, and reassembled at the client.
Definition **Server Push (HTTP/2)** — the server can send multiple responses for a single client request, pushing objects the client will need before it explicitly requests them, by analyzing the base HTML page.
Worked Example — HTTP/2 Framing Web page: 1 large video clip (1000 frames) + 8 small objects (2 frames each = 16 frames total).

- **Without interleaving (HTTP/1.1):** video sends all 1000 frames first; small objects begin at frame 1001. Total frame times to send all

small objects: **1016**.

- **With HTTP/2 interleaving:** frame 1 of video → frame 1 of each small object (8 frames) → frame 2 of video → frame 2 of each small object (8 frames). All small objects sent after: 2 + (2×8) = **18** frame times total.

Intuition HTTP/2 solves HOL blocking not by opening multiple TCP connections (which wastes resources and "cheats" TCP's fair share), but by breaking messages into frames and interleaving them — achieving parallelism within a **single** TCP connection.

Definition **HTTP/3** — designed to operate over **QUIC**, a new "transport" protocol implemented in the application layer over UDP. Features: message multiplexing, per-stream flow control, low-latency connection establishment. As of 2020, not yet fully standardized.

HTTP Request Message Format

Structure

- **Request line**: method field + URL field + HTTP version field
- **Header lines**: `Host:`, `Connection:`, `User-agent:`, `Accept-language:`, etc.
- Blank line (CR+LF)
- **Entity body** (empty for GET; used with POST)

```
GET /somedir/page.html HTTP/1.1
Host: www.someschool.edu
Connection: close
User-agent: Mozilla/5.0
Accept-language: fr
```

METHOD	PURPOSE
`GET`	Request an object identified by URL. Entity body empty.
`POST`	Submit form data in the entity body. Server returns a page depending on what the user entered.
`HEAD`	Like GET but server returns only headers, not the object. Used for debugging.
`PUT`	Upload an object to a specific path on a Web server.
`DELETE`	Delete an object on a Web server.

Watch Out A form using GET puts input data directly in the URL: `www.somesite.com/animalsearch?monkeys&bananas`. The entity body is **NOT** used.

HTTP Response Message Format

Structure

- **Status line**: protocol version + status code + status message
- **Header lines**: `Connection:`, `Date:`, `Server:`, `Last-Modified:`, `Content-Length:`, `Content-Type:`
- Blank line
- **Entity body** (the requested object)

```
HTTP/1.1 200 OK
Connection: close
Date: Tue, 18 Aug 2015 15:44:04 GMT
Server: Apache/2.2.3 (CentOS)
Last-Modified: Tue, 18 Aug 2015 15:11:03 GMT
Content-Length: 6821
Content-Type: text/html
(data data data data data ...)
```

STATUS CODE	MEANING
`200 OK`	Request succeeded; information returned in response
`301 Moved Permanently`	Object permanently moved; new URL in `Location:` header
`400 Bad Request`	Server could not understand the request
`404 Not Found`	Requested document does not exist on this server
`304 Not Modified`	Conditional GET response — cache can use its cached copy
`505 HTTP Version Not Supported`	—

Watch Out `Date:` is when the server **sends** the response, NOT when the object was last modified. `Last-Modified:` is when the object was created or last modified — the critical field for caching.

Watch Out `Content-Type:` is the **authoritative** indicator of object type — not the file extension.

2.3 Electronic Mail in the Internet

Three Major Components of the Internet Mail System

- **User agents** — let users read, reply, forward, save, and compose messages (Outlook, Apple Mail, Gmail)
- **Mail servers** — core of e-mail infrastructure. Each recipient has a mailbox in a mail server. Mail servers manage mailboxes and run both client and server sides of SMTP.
- **SMTP** — the principal application-layer protocol for Internet e-mail

Definition **SMTP (Simple Mail Transfer Protocol)** [RFC 5321] — uses TCP (port 25) for reliable transfer from sender's mail server to recipient's mail server. A **push** protocol — pushes mail to the receiving server. Older than HTTP (original RFC dates to 1982); restricts the message body to **7-bit ASCII** (binary data must be encoded before sending, decoded after receipt).

Definition **IMAP (Internet Mail Access Protocol)** [RFC 3501] — a **pull** protocol used by mail clients (e.g., Outlook) to retrieve e-mail from the mail server. Allows managing folders maintained on the server.

Watch Out — Why Two-Step Mail Delivery? Alice's UA sends to **her own** mail server first (not directly to Bob's), because her server can repeatedly retry sending to Bob's server (every 30 minutes for several days) if Bob's server is down. Alice's UA cannot do this itself.

SMTP	HTTP
Both use persistent connections	
Push protocol	**Pull** protocol
Puts **all objects in one message**	Encapsulates each object in its own response message
Requires **7-bit ASCII** body	Does not require this

Worked Example — SMTP Protocol Transcript Commands: `HELO`, `MAIL FROM`, `RCPT TO`, `DATA`, `QUIT`. Message ends with a line containing only a period (`CRLF.CRLF`).

```
S: 220 hamburger.edu
C: HELO crepes.fr
S: 250 Hello crepes.fr, pleased to meet you
C: MAIL FROM: <alice@crepes.fr>
S: 250 alice@crepes.fr ... Sender ok
C: RCPT TO: <bob@hamburger.edu>
S: 250 bob@hamburger.edu ... Recipient ok
C: DATA
S: 354 Enter mail, end with "." on a line by itself
C: Do you like ketchup?
C: How about pickles?
C: .
S: 250 Message accepted for delivery
C: QUIT
S: 221 hamburger.edu closing connection
```

SMTP uses **persistent connections**: if a sending server has multiple messages for the same receiving server, all are sent over the same TCP connection.

Mail Message Format (RFC 5322)

```
From: alice@crepes.fr
To: bob@hamburger.edu
Subject: Searching for the meaning of life.
(blank line)
(message body in ASCII)
```

Every header must have a `From:` and `To:` line; `Subject:` is optional.

Watch Out `From:` and `To:` in the mail message **header** (RFC 5322) are different from `MAIL FROM:` and `RCPT TO:` in the SMTP **commands** (RFC 5321). The commands are part of the handshaking protocol; the headers are part of the message itself.

Mail Access Protocol Flow (Alice → Bob) `Alice's UA →[SMTP or HTTP]→ Alice's mail server →[SMTP]→ Bob's mail server →[HTTP or IMAP]→ Bob's UA`

Alice's UA sends to her own mail server (not directly to Bob's), since her server can queue and retry. Bob's UA pulls mail from his server using HTTP (Web-based, e.g., Gmail) or IMAP.

2.4 DNS — The Internet's Directory Service

Definition **DNS (Domain Name System)** — (1) a distributed database implemented in a hierarchy of DNS servers, and (2) an application-layer protocol that lets hosts query the distributed database. DNS servers often run BIND software. DNS runs over **UDP, port 53**.
Definition **Hostname** — a mnemonic identifier for a host (e.g., `www.facebook.com`); variable-length, difficult for routers to process. **IP address** — 4 bytes (e.g., `121.7.106.83`), hierarchically structured — scanning left to right gives more specific location information.

What DNS Enables

- **Host aliasing** — a host with a complicated canonical hostname can have simpler alias hostnames (e.g., `relay1.west-coast.enterprise.com` aliasing to `enterprise.com`)
- **Mail server aliasing** — a company's mail server and Web server can have identical alias hostnames via **MX records**
- **Load distribution** — DNS distributes traffic among replicated servers by returning a set of IP addresses for one alias and rotating the ordering in each reply

The DNS Server Hierarchy

- **Root DNS servers** — >1000 instances worldwide; copies of 13 different root servers managed by 12 organizations, coordinated through IANA. Provide IP addresses of TLD servers.
- **TLD (Top-Level Domain) servers** — one per top-level domain (com, org, net, edu, gov, country codes). Maintained by companies like Verisign (com) and Educause (edu). Provide IP addresses of authoritative servers.
- **Authoritative DNS servers** — every organization with publicly accessible hosts must maintain these. House DNS records mapping host names to IP addresses. Orgs can implement their own or pay a service provider.
- **Local DNS server** (default name server) — provided by an ISP (typically via DHCP). Not part of the DNS hierarchy but central to DNS architecture; acts as a proxy, forwarding queries into the hierarchy. Typically "close to" the host.

Query Types

- **Recursive query** — asks the DNS server to obtain the mapping on the requester's behalf (e.g., host → local DNS server)
- **Iterative query** — the server replies with the name of the next server to contact; the requester makes all subsequent queries itself. Remaining queries (local DNS → root, TLD, authoritative) are typically iterative.

Definition **DNS Caching** — when a DNS server receives a reply, it can cache the mapping in local memory for a period (often ~2 days). Reduces query traffic and delay. Local DNS servers cache TLD server IPs, allowing root servers to be bypassed for most queries.

DNS Resource Records (RRs)

Format: `(Name, Value, Type, TTL)`. **TTL** — time to live — determines when a resource should be removed from a cache.

TYPE	NAME	VALUE	EXAMPLE
A	Hostname	IP address	(relay1.bar.foo.com, 145.37.93.126, A)
NS	Domain	Hostname of authoritative DNS server	(foo.com, dns.foo.com, NS)
CNAME	Alias hostname	Canonical hostname	(foo.com, relay1.bar.foo.com, CNAME)
MX	Mail alias	Canonical name of mail server	(foo.com, mail.bar.foo.com, MX)

Key Rule If a DNS server is **authoritative** for a hostname, it contains a Type A record for that hostname. If **not** authoritative, it contains a Type NS record pointing to the authoritative server, plus a Type A record for that NS server.

DNS Message Format

Header Section (12 bytes)

- 16-bit identification (matches reply to query)
- Flags: query/reply (0/1), authoritative flag, recursion-desired, recursion-available
- Four count fields (number of questions, answer RRs, authority RRs, additional RRs)

Sections

1. **Questions** — name being queried + type field
2. **Answers** — resource records for the queried name (may have multiple RRs for replicated servers)
3. **Authority** — records of other authoritative servers
4. **Additional information** — other helpful records (e.g., Type A records for mail servers mentioned in MX answers)

Worked Example — DNS Lookup Host `cse.nyu.edu` wants the IP address of `gaia.cs.umass.edu`:

1. `cse.nyu.edu` sends a DNS query to local DNS server `dns.nyu.edu` (recursive)
2. `dns.nyu.edu` → root DNS server (iterative)
3. Root → returns IP addresses of `edu` TLD servers
4. `dns.nyu.edu` → TLD server
5. TLD server → returns IP address of `dns.umass.edu` (authoritative for umass.edu)
6. `dns.nyu.edu` → `dns.umass.edu`
7. `dns.umass.edu` → returns IP address of `gaia.cs.umass.edu`

8. `dns.nyu.edu` returns the IP address to `cse.nyu.edu`

Total messages sent: 8 (4 queries + 4 replies)

Intuition DNS uses a hierarchy because a centralized design would create: a single point of failure, unmanageable traffic volume, high latency for distant clients, and a maintenance nightmare.

Worked Example — Inserting DNS Records (Registering networkutopia.com)

1. Provide the registrar with names and IPs of primary and secondary authoritative DNS servers: `dns1.networkutopia.com` (212.212.212.1), `dns2.networkutopia.com`
2. Registrar inserts into TLD `com` servers: `(networkutopia.com, dns1.networkutopia.com, NS)` and `(dns1.networkutopia.com, 212.212.212.1, A)`
3. You add a Type A record for `www.networkutopia.com` and a Type MX record for `mail.networkutopia.com` in your own authoritative DNS servers

2.5 Peer-to-Peer File Distribution

Definition **P2P File Distribution** — distributing a large file from a single server to many peers. Each peer can redistribute any portion of the file it has received to other peers, assisting the server.

Definition **Distribution Time** — the time it takes to get a copy of the file to all N peers.

Variables

- F = file size in bits
- N = number of peers
- u_s = server upload rate
- u_i = upload rate of peer i
- d_i = download rate of peer i
- d_{min} = minimum download rate among all peers = min{d_1, d_2, …, d_N}

Formula — Client-Server Distribution Time (Eq. 2.1, lower bound) $D_{cs} = \max(NF/u_s, F/d_{min})$

For large N, dominated by NF/u_s → increases **linearly** with N.

Formula — P2P Distribution Time (Eq. 2.2/2.3, lower bound) $D_{P2P} = \max(F/u_s, F/d_{min}, NF/(u_s + \Sigma u_i))$

where $\Sigma u_i = u_1 + u_2 + \ldots + u_N$

Intuition In client-server, only the server uploads to all N peers. In P2P, total upload capacity is $u_s + \Sigma u_i$ — each new peer adds upload capacity, enabling **self-scalability**. P2P minimum distribution time stays under 1 hour (for Figure 2.23's parameters) regardless of N.

BitTorrent

Definition **Torrent** — the collection of all peers participating in the distribution of a particular file. **Chunk** — equal-size piece of file downloaded from peers; typical size: **256 KBytes**. **Tracker** — infrastructure node in each torrent; tracks which peers are registered/participating.

Joining BitTorrent

1. New peer (Alice) registers with the tracker
2. Tracker randomly selects ~50 participating peers and sends their IPs to Alice
3. Alice attempts concurrent TCP connections with all 50 → successful ones become **neighboring peers**

Chunk Selection — "Rarest First" Alice requests the chunks that are rarest among her neighbors first, so the rarest chunks get redistributed more quickly, equalizing chunk distribution across the torrent.

Peer Selection — Tit-for-Tat

- Alice measures the rate at which she receives bits from each neighbor
- She identifies the **top-4** peers feeding her at the highest rate and sends chunks to those same 4 (unchoked peers); recalculated every **10 seconds**
- Every **30 seconds**, Alice picks one additional random neighbor and sends it chunks (**optimistically unchoked** peer) — lets new peers get chunks and potentially become top-4
- All other neighbors are **choked** — receive nothing from Alice

Intuition Tit-for-tat creates an incentive for peers to upload: only peers who upload to Alice receive chunks from Alice. Random selection (optimistic unchoking) prevents the system from locking into a fixed set of trading partners and helps new peers bootstrap.

Definition **DHT (Distributed Hash Table)** — a simple database with records distributed over P2P peers. Widely implemented (e.g., in BitTorrent).

2.6 Video Streaming and Content Distribution Networks

Streaming video accounts for ~80% of Internet traffic by many 2020 estimates.

Definition **Video** — a sequence of images displayed at a constant rate (24 or 30 images/sec). Can be compressed, trading off quality vs. bit rate. Compressed Internet video: 100 kbps (low quality) to >4 Mbps (HD); 4K: >10 Mbps.

Most important performance measure for streaming video: **average end-to-end throughput** — must be at least as large as the video's bit rate.

Definition **HTTP streaming (basic)** — video stored on an HTTP server as an ordinary file. Client issues GET; bytes accumulate in a client buffer; playback begins once the buffer exceeds a threshold. Limitation: all clients receive the same encoding regardless of bandwidth variations.
Definition **DASH (Dynamic Adaptive Streaming over HTTP)** — video encoded into several versions at different bit rates/quality levels. Client dynamically requests chunks (a few seconds each) via HTTP GET, selecting higher-rate chunks when bandwidth is high and lower-rate when low. Each version stored with a different URL; the server has a **manifest file** listing URLs and bit rates for each version.
Definition **CDN (Content Distribution Network)** — manages servers in multiple geographically distributed locations, stores copies of content, and directs each user request to the location providing the best experience.

Two CDN Placement Philosophies

- **Enter Deep** (pioneered by Akamai) — deploy server clusters inside access ISPs worldwide; thousands of locations. Goal: minimize hops between user and CDN. Challenging to maintain.
- **Bring Home** (e.g., Limelight) — place large clusters at fewer sites (~tens) at IXPs. Lower maintenance overhead but possibly higher delay.

Definition **CDN Pull Strategy** — if a client requests a video not in a cluster, the cluster retrieves it from a central repository or another cluster, stores a local copy while streaming it, and removes infrequently requested videos when storage fills.

Worked Example — CDN Operation (DNS-based redirection, NetCinema/KingCDN)

1. User visits NetCinema, clicks a video URL (`http://video.netcinema.com/6Y7B23V`)
2. User's host sends a DNS query for `video.netcinema.com`
3. LDNS relays to NetCinema's authoritative DNS → returns `a1105.kingcdn.com` (not an IP address!)
4. LDNS sends a second query for `a1105.kingcdn.com` to KingCDN's DNS → returns the IP of a content server
5. LDNS forwards the IP to the user's host
6. Client establishes a direct TCP connection with the KingCDN server → HTTP GET → video served (possibly via DASH)

Cluster Selection Strategies

1. **Geographically closest cluster** (geo-location databases) — simple but ignores network conditions and remotely located LDNSs
2. **Real-time measurements** (delay/loss probes to LDNSs) — better, but many LDNSs block probe responses

Netflix vs. YouTube Comparison

FEATURE	NETFLIX	YOUTUBE
CDN type	Private CDN (built own)	Private CDN (Google)
Cluster locations	IXPs + residential ISPs (>200 IXPs)	IXPs + ISPs + mega data centers
Cache strategy	**Push** (off-peak hours)	**Pull** (demand-based)
Streaming	Proprietary DASH with byte-range HTTP GET	HTTP streaming (manual version selection)
DNS redirect	No — Netflix software (Amazon cloud) directly assigns CDN server	Yes
Chunk size	~4 seconds	Uses HTTP byte-range request
Content processing	Amazon cloud ingests/encodes/uploads to CDN	Within Google data centers

Netflix Architecture Components

1. **Amazon cloud** — user registration/login, billing, movie catalogue, recommendation system, content ingestion/processing, upload to CDN
2. **Netflix private CDN** — IXP racks (tens of servers, ~100 TB storage, entire Netflix library) and ISP racks

Watch Out Netflix uses **push** caching, not pull — content is pushed to CDN servers during off-peak hours. YouTube uses **pull** caching. Netflix does **not** use DNS redirect to connect clients to CDN servers; instead, the Netflix software directly tells the client which CDN server to use.

2.7 Socket Programming: Creating Network Applications

Definition **Socket Programming** — creating network applications using the socket API. The developer's main task: write code for both client and server programs.

Open vs. Proprietary Applications

- **Open application** — implements a protocol specified in an RFC; client and server must conform to its rules. Example: HTTP client (Chrome) + HTTP server (Apache).
- **Proprietary application** — uses an unpublished application-layer protocol. Developer has complete control, but other developers cannot interoperate.

2.7.1 UDP Socket Programming

In UDP, before sending, you attach the destination address (IP + port) to the packet. The receiving process retrieves the packet through its socket and inspects the contents.

UDPClient.py — Full Code

```
from socket import *
serverName = 'hostname'
serverPort = 12000
clientSocket = socket(AF_INET, SOCK_DGRAM)
message = input('Input lowercase sentence:')
clientSocket.sendto(message.encode(), (serverName, serverPort))
modifiedMessage, serverAddress = clientSocket.recvfrom(2048)
print(modifiedMessage.decode())
clientSocket.close()
```

Line-by-Line

- `socket(AF_INET, SOCK_DGRAM)` — creates a UDP socket. `AF_INET` = IPv4; `SOCK_DGRAM` = UDP.
- `message.encode()` — converts string to bytes (required for sockets)
- `clientSocket.sendto(message.encode(), (serverName, serverPort))` — attaches destination address and sends; source address attached automatically by the OS
- `clientSocket.recvfrom(2048)` — returns `(data, serverAddress)`; buffer size 2048

Watch Out Port number is **NOT** specified for the client socket — the OS assigns it automatically.

UDPServer.py — Full Code

```
from socket import *
serverPort = 12000
serverSocket = socket(AF_INET, SOCK_DGRAM)
serverSocket.bind(('', serverPort))
print("The server is ready to receive")
while True:
    message, clientAddress = serverSocket.recvfrom(2048)
    modifiedMessage = message.decode().upper()
    serverSocket.sendto(modifiedMessage.encode(), clientAddress)
```

Key difference from the client: `serverSocket.bind(('', serverPort))` explicitly assigns port 12000 to the server socket, so any packet sent to that port at this server's IP is directed here. The server runs an infinite loop.

2.7.2 TCP Socket Programming

In TCP, a connection must be established before data can flow. The server side uses **two** sockets.

Two Server-Side Sockets

1. **Welcoming socket** (`serverSocket`) — accepts initial contact from any client
2. **Connection socket** (`connectionSocket`) — created for each specific client after the handshake; dedicated to that client

TCPClient.py — Full Code

```
from socket import *
serverName = 'servername'
serverPort = 12000
clientSocket = socket(AF_INET, SOCK_STREAM)
clientSocket.connect((serverName, serverPort))
sentence = input('Input lowercase sentence:')
clientSocket.send(sentence.encode())
modifiedSentence = clientSocket.recv(1024)
print('From Server: ', modifiedSentence.decode())
clientSocket.close()
```

Key Differences from UDP

- `SOCK_STREAM` instead of `SOCK_DGRAM` → TCP socket
- `clientSocket.connect((serverName, serverPort))` — performs the three-way handshake and establishes the connection; invisible to the application
- `clientSocket.send(sentence.encode())` — no need to attach a destination address; the TCP connection already knows where to go

TCPServer.py — Full Code

```
from socket import *
serverPort = 12000
serverSocket = socket(AF_INET, SOCK_STREAM)
serverSocket.bind(('', serverPort))
serverSocket.listen(1)
print('The server is ready to receive')
while True:
    connectionSocket, addr = serverSocket.accept()
    sentence = connectionSocket.recv(1024).decode()
    capitalizedSentence = sentence.upper()
    connectionSocket.send(capitalizedSentence.encode())
    connectionSocket.close()
```

Key Elements

- `serverSocket.listen(1)` — listens for connection requests; parameter = max number of queued connections (at least 1)
- `serverSocket.accept()` — creates a new socket (`connectionSocket`) dedicated to this client; the three-way handshake completes here
- `connectionSocket.close()` — closes the connection to this client; `serverSocket` stays open for the next one

UDP needs 1 socket; TCP needs 2 (welcoming + connection). For n simultaneous TCP connections, the server needs **n+1** sockets (1 welcoming + n connection sockets).

FEATURE	UDP	TCP
Socket type	`SOCK_DGRAM`	`SOCK_STREAM`
Connection	No connection setup	Must call `.connect()`
Server sockets needed	1	2 (welcoming + connection)
Address attachment	Explicit (`sendto`)	Automatic (via connection)
Data delivery	No guarantee	Reliable, in-order
Server start order	Client can start first	Server must start first

Watch Out For TCP client-server: the **server must already be running** before the client attempts to connect (server must be listening). For UDP, the client may execute **before** the server, since there's no connection setup — the packet simply gets lost if no server is listening.

★ Quick Reference Sheet

All Formulas

FORMULA	DESCRIPTION
Response time (non-persistent HTTP) = 2×RTT + file transmission time	Time from click to full page receipt (one object)
Traffic intensity = (request rate × object size) / link bandwidth	How loaded a link is
Avg delay (with cache) = $h \times (\text{delay}_{\text{cache}}) + (1-h) \times (\text{delay}_{\text{origin}})$	Average response time with Web cache hit rate h
$D_{cs} = \max(NF/u_s,\ F/d_{min})$	Client-server file distribution time
$D_{P2P} = \max(F/u_s,\ F/d_{min},\ NF/(u_s+\Sigma u_i))$	P2P file distribution time

All Key Definitions

TERM	DEFINITION
Socket	Software interface (API) between application layer and transport layer
Client process	Process that initiates communication at the start of a session
Server process	Process that waits to be contacted to begin the session
HTTP	HyperText Transfer Protocol; Web's application-layer protocol; stateless; runs over TCP port 80
Stateless protocol	Server maintains no information about clients between requests
RTT	Round-Trip Time; time for a small packet to travel client→server→client
Persistent connection	All request/response pairs sent over the same TCP connection
Non-persistent connection	Each request/response pair sent over a separate TCP connection (HTTP/1.0)
Pipelining	Client sends requests back-to-back without waiting for replies (HTTP/1.1 persistent connections)
Cookie	Set-cookie/Cookie headers + browser cookie file + server database, creating user sessions on stateless HTTP
Web cache / Proxy server	Network entity satisfying HTTP requests on behalf of an origin server; acts as both client and server
Hit rate	Fraction of requests satisfied by a cache (typically 0.2–0.7)
Conditional GET	HTTP GET with If-Modified-Since header; server returns 304 Not Modified if unchanged
HOL blocking	Head of Line blocking in HTTP/1.1: large objects block small objects on the same TCP connection
HTTP/2 framing	Breaking HTTP messages into interleaved binary-encoded frames to eliminate HOL blocking
Server push	HTTP/2 feature allowing the server to send objects before the client requests them
SMTP	Simple Mail Transfer Protocol; push protocol; TCP port 25; requires 7-bit ASCII body
IMAP	Internet Mail Access Protocol; pull protocol; allows managing mail folders on server
DNS	Domain Name System; distributed hierarchical database translating hostnames to IP addresses; UDP port 53
Root DNS server	Top of DNS hierarchy; >1000 instances; provides TLD server IP addresses
TLD server	Top-Level Domain server; one per top-level domain; provides authoritative server addresses
Authoritative DNS server	Stores DNS records for an organization's publicly accessible hosts
Local DNS server	ISP-provided DNS proxy; forwards queries into the hierarchy; not strictly part of the hierarchy
Recursive query	DNS query asking the server to fully resolve the hostname on the requester's behalf
Iterative query	DNS query where the server returns the next server to contact; requester makes each subsequent query
Resource Record (RR)	DNS database entry: (Name, Value, Type, TTL)
DASH	Dynamic Adaptive Streaming over HTTP; client dynamically selects video quality based on bandwidth
Manifest file	DASH file listing URLs and bit rates for each video version
CDN	Content Distribution Network; geographically distributed servers storing copies of content
Enter Deep	CDN philosophy: deploy clusters inside access ISPs worldwide (Akamai)
Bring Home	CDN philosophy: fewer large clusters at IXPs (Limelight)
Torrent	Collection of all peers participating in BitTorrent distribution of a file

Chunk	Equal-size piece of file in BitTorrent (typical: 256 KBytes)
Tracker	Infrastructure node in a BitTorrent torrent; maintains list of participating peers
Rarest first	BitTorrent strategy: request chunks rarest among neighbors first
Tit-for-tat	BitTorrent incentive: send chunks to the top-4 peers feeding you fastest; optimistically unchoke 1 random peer every 30 sec
TLS	Transport Layer Security; application-layer enhancement to TCP providing encryption, integrity, authentication
Self-scalability (P2P)	Each new peer adds upload capacity as well as demand, so distribution time doesn't grow linearly

Example Problem Types Covered

- HTTP response time calculation — non-persistent vs. persistent HTTP using RTT
- Traffic intensity calculation — determining whether a link is overloaded
- Web cache average delay calculation — response time improvement with hit rate h
- Conditional GET trace analysis — identifying 304 Not Modified responses
- HTTP message parsing — identifying method, URL, version, status code, header fields from raw messages
- SMTP dialogue — writing or analyzing client-server SMTP command/reply exchanges
- DNS query chain tracing — counting messages sent for iterative vs. recursive lookups
- DNS record identification — choosing the appropriate RR type (A, NS, CNAME, MX) for a scenario
- Client-server distribution time — applying $D_{cs} = \max(NF/u_s, F/d_{min})$
- P2P distribution time — applying $D_{P2P} = \max(F/u_s, F/d_{min}, NF/(u_s+\Sigma u_i))$
- BitTorrent unchoke/tit-for-tat logic — determining which peers Alice unchokes and why
- HTTP/2 frame interleaving — calculating frame times with and without interleaving
- UDP socket programming — reading/writing Python UDPClient.py and UDPServer.py
- TCP socket programming — reading/writing Python TCPClient.py and TCPServer.py
- CDN DNS redirection flow — tracing the steps for a CDN to serve video to a user
- Application protocol classification — determining whether to use UDP or TCP for a given application